

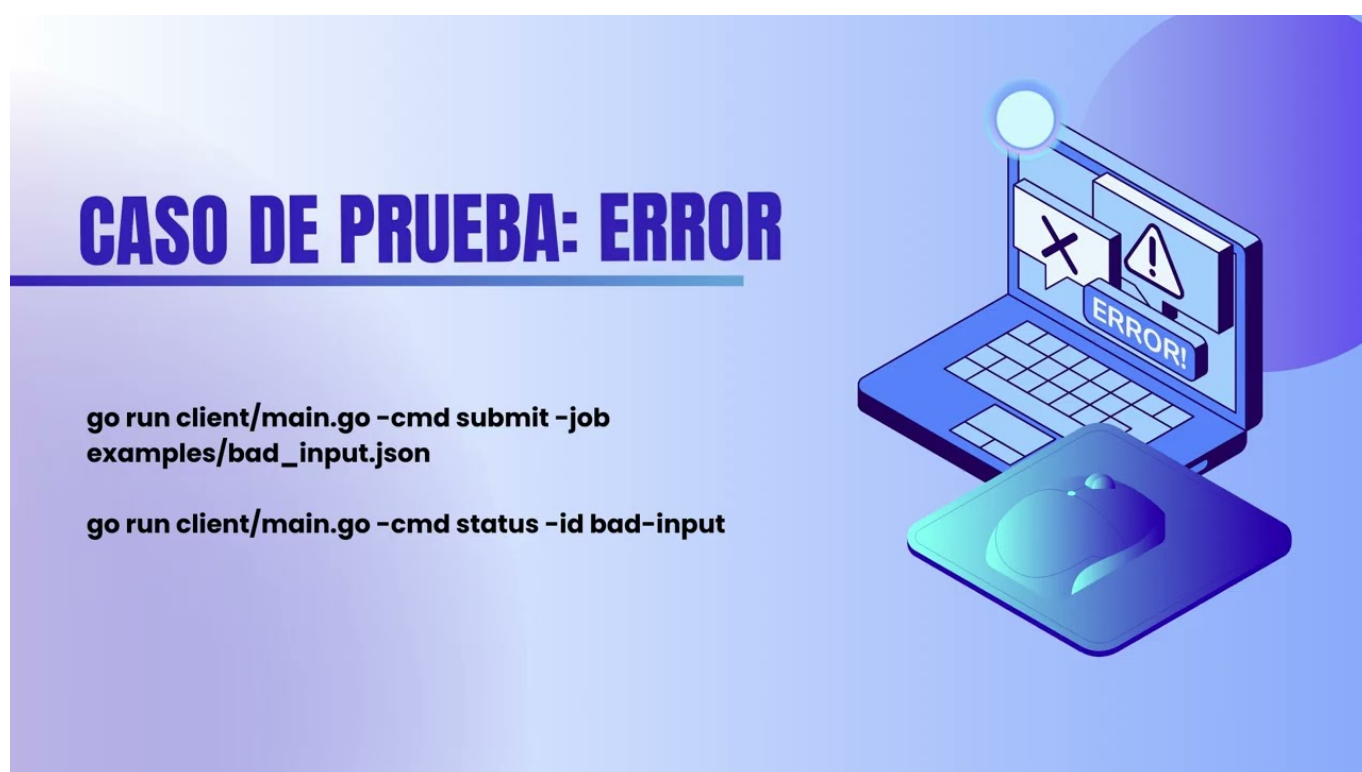
Mini-Spark - Guía de Uso Completa

Esta guía cubre todo lo necesario para compilar, ejecutar, probar y detener el sistema Mini-Spark.

Requisitos Previos

- **Docker** y **Docker Compose** instalados
 - **Go 1.21+** (solo si vas a compilar manualmente)
 - **GNU Make** (opcional, para usar Makefile)
-

Enlace a video demo



[Ver demo en YouTube](#)

Opción 1: Compilación Manual (Sin Make)

Paso 1: Compilar el Master

```
cd master
go build -o master main.go scheduler.go
cd ..
```

Paso 2: Compilar los Workers

```
cd worker
go build -o worker main.go executor.go
cd ..
```

Paso 3: Compilar el Cliente CLI

```
cd client
go build -o client main.go
cd ..
```

Resultado:

- `master/master` (o `master.exe` en Windows)
- `worker/worker` (o `worker.exe` en Windows)
- `client/client` (o `client.exe` en Windows)

Opción 2: Compilación con Makefile

Si tienes GNU Make instalado, puedes usar estos comandos:

Ver todos los comandos disponibles

```
make help
```

Salida:

```
Mini-Spark - Sistema de Procesamiento Distribuido

Uso:
  make <target>

General
  help          Muestra esta ayuda

Build
  build         Compila todos los componentes localmente
  build-master  Compila solo el master
  build-worker  Compila solo el worker
  build-client  Compila el cliente CLI

Docker
  docker-build  Construye las imágenes Docker sin cache
  up            Inicia el cluster (1 master + 3 workers)
  down         Detiene y elimina el cluster
  restart       Reinicia el cluster completo
```

Logs y Monitoreo

logs	Muestra logs de todos los contenedores
logs-master	Muestra logs solo del master
logs-workers	Muestra logs de todos los workers
status	Muestra el estado del cluster

Testing

test	Ejecuta pruebas unitarias
test-integration	Ejecuta pruebas de integración

Limpieza

clean	Limpia binarios y datos temporales
clean-all	Limpieza completa incluyendo imágenes Docker

Demo

demo	Inicia demo interactivo del sistema
------	-------------------------------------

Desarrollo

dev-master	Ejecuta el master en modo desarrollo
dev-worker	Ejecuta un worker en modo desarrollo
fmt	Formatea el código Go
lint	Ejecuta linter en el código

Compilar todo el proyecto

```
make build
```

Compilar componentes individuales

```
make build-master    # Solo el Master
make build-worker    # Solo los Workers
make build-client    # Solo el Cliente CLI
```

Iniciar el Cluster

Opción A: Con Makefile (Recomendado)

```
make up
```

Esto automáticamente:

1. Crea los directorios necesarios (`app/data/`, `app/results/`, `app/storage/`, `app/temp/`)
2. Construye las imágenes Docker

3. Inicia el cluster (1 master + 3 workers)
4. Espera 5 segundos
5. Muestra el estado del cluster

Salida esperada:

```
Iniciando cluster Mini-Spark...
✓ Network proy_2_so_minispark-net    Created
✓ Container minispark-master         Healthy
✓ Container minispark-worker1        Started
✓ Container minispark-worker2        Started
✓ Container minispark-worker3        Started
✓ Cluster iniciado
Master API: http://localhost:8080
Health check: http://localhost:8080/health

Estado del Cluster:
NAME                STATUS              PORTS
minispark-master    Up (healthy)        0.0.0.0:8080->8080/tcp
minispark-worker1    Up                  8081/tcp
minispark-worker2    Up                  8081/tcp
minispark-worker3    Up                  8081/tcp
```

Opción B: Con Docker Compose directamente

```
docker compose up -d --build
```

Salida esperada:

```
✓ Network proy_2_so_minispark-net    Created
✓ Container minispark-master         Healthy
✓ Container minispark-worker1        Started
✓ Container minispark-worker2        Started
✓ Container minispark-worker3        Started
```

Verificar estado del cluster**Con Make:**

```
make status
```

Con Docker Compose:

```
docker compose ps
```

Salida esperada:

NAME	IMAGE	STATUS	
minispark-master	proy_2_so-master	Up (healthy)	0.0.0.0:8080->8080/tcp
minispark-worker1	proy_2_so-worker1	Up	8081/tcp
minispark-worker2	proy_2_so-worker2	Up	8081/tcp
minispark-worker3	proy_2_so-worker3	Up	8081/tcp

Health check del cluster

Opción A: Usar curl directo

```
curl http://localhost:8080/health
```

Salida:

```
{
  "master_id": "master-1764268682",
  "status": "healthy",
  "timestamp": "2025-11-27T18:38:22Z",
  "workers_total": 3,
  "workers_up": 3
}
```

Opción B: Usar el CLI

```
./client.exe -cmd health
```

Salida:

```
┌── CLUSTER HEALTH ──┐
✓ Master: healthy
✓ Workers: 3/3 online
└──────────────────┘
```

 Ver Logs del Sistema

Con Makefile

```
# Ver todos los logs en tiempo real
make logs

# Ver logs solo del Master
make logs-master

# Ver logs solo de Workers
make logs-workers
```

Con Docker Compose

```
# Ver todos los logs en tiempo real
docker compose logs -f

# Ver logs de un componente específico
docker compose logs -f master

# Ver logs de todos los workers
docker compose logs -f worker1 worker2 worker3

# Ver un worker específico
docker compose logs -f worker1

# Ver últimas 50 líneas de logs
docker compose logs master | tail -50

# Buscar en logs
docker compose logs master | grep "Job"
```

Logs del Master

Los logs del Master muestran:

- Registro de workers
- Creación de jobs
- Asignación de tareas
- Completitud de tareas
- Auto-guardado de estado

Ejemplo:

```
[INFO] [MASTER] Registrando worker: worker-worker1-1764268688
[INFO] [MASTER] Creando nuevo job: aggregate-multi-function
[INFO] [MASTER] Iniciando scheduling para job: aggregate-multi-function
[INFO] [MASTER] Actualización de tarea aggregate-multi-function-task-0-part-0:
COMPLETED
[INFO] [MASTER] ✓ Job aggregate-multi-function completado exitosamente (6 tareas)
```

```
[INFO] [MASTER] Estado guardado: state-1764268712.json (5 jobs, 38 tasks, 3 workers)
```

Logs de Workers

Los logs de workers muestran:

- Heartbeats enviados
- Tareas recibidas
- Ejecución de operadores
- Métricas (CPU, memoria)

Detener y Reiniciar el Cluster

Detener el cluster

Con Make:

```
make down
```

Con Docker Compose:

```
docker compose down
```

Salida:

```
✓ Container minispark-worker3      Removed
✓ Container minispark-worker2      Removed
✓ Container minispark-worker1      Removed
✓ Container minispark-master       Removed
✓ Network proy_2_so_minispark-net  Removed
```

Reiniciar el cluster

Con Make:

```
make restart
```

Con Docker Compose:

```
docker compose restart
```

Detener y eliminar volúmenes

Con Make:

```
make clean-all
```

Con Docker Compose:

```
docker compose down -v
```

⚠ **ADVERTENCIA:** Esto eliminará todos los datos en `app/results/` y `app/storage/`.

Limpieza de Archivos

Limpiar binarios compilados y datos temporales

Con Make:

```
make clean
```

Esto elimina:

- Binarios compilados (`master/master`, `worker/worker`)
- Archivos en `app/results/`
- Archivos en `app/storage/`

Limpieza completa (incluyendo imágenes Docker)

Con Make:

```
make clean-all
```

Esto elimina:

- Todo lo de `make clean`
- Contenedores Docker
- Imágenes Docker
- Volúmenes Docker

Manual:


```
# Limpiar binarios
rm -f master/master worker/worker client/client

# Limpiar resultados y storage
rm -rf app/results/* app/storage/*

# Limpiar archivos temporales
rm -rf app/temp/*
```

Uso del Cliente CLI

El cliente CLI es la forma principal de interactuar con el sistema.

Comandos Disponibles

Comando	Descripción	Ejemplo
health	Verificar estado del cluster	<code>./client.exe -cmd health</code>
submit	Enviar un job para ejecución	<code>./client.exe -cmd submit -job examples/wordcount.json</code>
status	Ver estado detallado de un job	<code>./client.exe -cmd status -id my-job-id</code>
list	Listar todos los jobs	<code>./client.exe -cmd list</code>

1. Verificar Cluster (health)

```
./client.exe -cmd health
```

Muestra:

- Estado del Master
- Número de workers activos (N/3)
- Timestamp actual

2. Listar Jobs (list)

```
./client.exe -cmd list
```

Salida:

```
┌── JOB LIST ──┐
```

Total: 5 jobs

JOB ID	STATUS	NODES	SUBMITTED
complete-pipeline-job	SUCCEEDED	4	08:26:36 27/11
aggregate-multi-function	SUCCEEDED	6	08:27:04 27/11
user-orders-join	SUCCEEDED	3	08:27:20 27/11
benchmark-10k-lines	SUCCEEDED	3	08:29:33 27/11
wordcount-example	RUNNING	2	08:30:15 27/11

3. Enviar un Job (submit)

Formato básico:

```
./client.exe -cmd submit -job <ruta-al-archivo.json>
```

Ejemplos:

```
# Job simple de agregación
./client.exe -cmd submit -job examples/aggregate_sales.json

# Pipeline completo (4 etapas)
./client.exe -cmd submit -job examples/complete-pipeline.json

# Join de dos datasets
./client.exe -cmd submit -job examples/join_job.json

# Benchmark de 10K líneas
./client.exe -cmd submit -job examples/benchmark_job.json
```

Salida:

```
┌ SUBMIT JOB ─┐
i Leyendo archivo: examples/aggregate_sales.json
i Job ID: aggregate-sales-job
i Nodos DAG: 3
└───────────┘

⚡ Enviando al Master... Done!
✓ Job enviado exitosamente
i Job ID: aggregate-sales-job

Tip: Para monitorear: ./client -cmd status -id aggregate-sales-job
```

4. Ver Estado de un Job (status)

```
./client.exe -cmd status -id aggregate-multi-function
```

Salida:

JOB STATUS

▸ Job: aggregate-multi-function

▸ Status: SUCCEEDED

▸ DAG: 6 nodes, 0 edges

🕒 Submitted: 18:38:33 27/11/2025

▶ Started: 18:38:33 27/11/2025

✓ Completed: 18:38:35 27/11/2025 (duration: 2.536s)

▸ Tasks: 6 total

COMPLETED

6 (100%)

Uso Directo de la API REST

El Master expone una API REST en <http://localhost:8080>.

Endpoints Disponibles

Método	Endpoint	Descripción
GET	/health	Health check del cluster
GET	/api/v1/jobs	Listar todos los jobs
GET	/api/v1/jobs/{id}	Detalles de un job específico
POST	/api/v1/jobs	Crear un nuevo job
POST	/api/v1/workers/register	Registrar un worker
POST	/api/v1/workers/heartbeat	Enviar heartbeat
POST	/api/v1/tasks/update	Actualizar estado de tarea

Ejemplos con curl

1. Health Check:

```
curl http://localhost:8080/health
```

2. Listar Jobs:

```
curl http://localhost:8080/api/v1/jobs | jq .
```

3. Detalles de un Job:

```
curl http://localhost:8080/api/v1/jobs/aggregate-multi-function | jq .
```

4. Enviar un Job:

```
curl -X POST http://localhost:8080/api/v1/jobs \  
-H "Content-Type: application/json" \  
-d @examples/aggregate_sales.json
```

Estructura de un Job (JSON)

Los jobs se definen en archivos JSON con el siguiente formato:

```
{  
  "id": "my-custom-job",  
  "name": "My Data Processing Job",  
  "dag": {  
    "nodes": [  
      {  
        "id": "read-input",  
        "operator": "read_csv",  
        "input_paths": ["input.csv"],  
        "output_path": "app/temp/data-read.csv",  
        "partitions": 3  
      },  
      {  
        "id": "transform",  
        "operator": "map",  
        "input_paths": ["app/temp/data-read.csv"],  
        "output_path": "app/temp/data-transformed.csv",  
        "partitions": 3,  
        "params": {  
          "function": "uppercase"  
        }  
      },  
      {  
        "id": "aggregate",  
        "operator": "reduce_by_key",  
        "input_paths": ["app/temp/data-transformed.csv"],  
        "output_path": "app/results/final-output.csv",
```

```

    "partitions": 1
  }
],
"edges": [
  {"from": "read-input", "to": "transform"},
  {"from": "transform", "to": "aggregate"}
]
}
}

```

Campos Principales

- **id**: Identificador único del job
- **name**: Nombre descriptivo (opcional)
- **dag**: Definición del grafo de ejecución
 - **nodes**: Lista de operadores a ejecutar
 - **id**: ID único del nodo
 - **operator**: Tipo de operador (ver sección siguiente)
 - **input_paths**: Archivos de entrada
 - **output_path**: Archivo de salida
 - **partitions**: Número de particiones paralelas
 - **params**: Parámetros específicos del operador
 - **edges**: Dependencias entre nodos
 - **from**: Nodo origen
 - **to**: Nodo destino

Operadores Disponibles

1. **read_csv** - Lectura de CSV

Lee un archivo CSV y lo particiona para procesamiento distribuido.

```

{
  "id": "read-data",
  "operator": "read_csv",
  "input_paths": ["input.csv"],
  "output_path": "app/temp/data-read.csv",
  "partitions": 3
}

```

2. **map** - Transformación 1-a-1

Aplica una función a cada registro.

Funciones disponibles:

- **lowercase**: Convierte a minúsculas

- **uppercase**: Convierte a MAYÚSCULAS

```
{
  "id": "transform",
  "operator": "map",
  "input_paths": ["app/temp/input.csv"],
  "output_path": "app/temp/output.csv",
  "partitions": 2,
  "params": {
    "function": "uppercase"
  }
}
```

3. **filter** - Filtrado Condicional

Filtra registros que cumplen una condición.

```
{
  "id": "filter-data",
  "operator": "filter",
  "input_paths": ["app/temp/input.csv"],
  "output_path": "app/temp/filtered.csv",
  "partitions": 2,
  "params": {
    "condition": "length > 0"
  }
}
```

4. **flat_map** - Expansión 1-a-N

Expande cada registro en múltiples registros.

Funciones disponibles:

- **split_words**: Tokeniza texto en palabras
- **split_delimiter**: Divide por delimitador personalizado
- **explode_array**: Expande arrays

```
{
  "id": "tokenize",
  "operator": "flat_map",
  "input_paths": ["app/temp/text.csv"],
  "output_path": "app/temp/words.csv",
  "partitions": 4,
  "params": {
    "function": "split_words"
  }
}
```

5. **reduce_by_key** - Agrupación y Reducción

Agrega registros por clave y cuenta ocurrencias.

```
{
  "id": "count-words",
  "operator": "reduce_by_key",
  "input_paths": ["app/temp/words.csv"],
  "output_path": "app/results/wordcount.csv",
  "partitions": 2
}
```

6. **aggregate** - Agregación Configurable

Aplica funciones de agregación (count, sum, avg, min, max).

```
{
  "id": "sum-sales",
  "operator": "aggregate",
  "input_paths": ["app/temp/sales.csv"],
  "output_path": "app/results/total-sales.csv",
  "partitions": 1,
  "params": {
    "function": "sum",
    "value_column": 1
  }
}
```

Funciones disponibles:

- **count**: Cuenta registros
- **sum**: Suma de columna numérica
- **avg**: Promedio
- **min**: Valor mínimo
- **max**: Valor máximo

7. **join** - Join de Datasets

Realiza inner join entre dos datasets por clave.

```
{
  "id": "join-users-orders",
  "operator": "join",
  "input_paths": ["app/temp/users.csv", "app/temp/orders.csv"],
  "output_path": "app/results/joined.csv",
  "partitions": 2,
}
```

```
"params": {  
  "join_key": "user_id",  
  "join_type": "inner"  
}
```

Límites de Timeout y Memoria

Puedes configurar límites de tiempo y memoria para cada tarea:

```
{  
  "id": "heavy-task",  
  "operator": "aggregate",  
  "input_paths": ["large-data.csv"],  
  "output_path": "results/output.csv",  
  "partitions": 4,  
  "params": {  
    "function": "sum",  
    "value_column": 1,  
    "timeout_sec": 300,  
    "max_memory_mb": 512  
  }  
}
```

- **timeout_sec**: Tiempo máximo de ejecución en segundos (0 = sin límite)
- **max_memory_mb**: Memoria máxima en MB (0 = sin límite)

Si una tarea excede estos límites, será marcada como FAILED y se reintentará.

Ejemplos de Jobs Incluidos

El proyecto incluye 5 ejemplos listos para usar en la carpeta `examples/`:

1. **aggregate_sales.json** - Agregación Simple

Calcula suma de ventas por ciudad.

```
./client.exe -cmd submit -job examples/aggregate_sales.json
```

2. **aggregate_multiple.json** - Múltiples Agregaciones

Ejecuta 5 funciones de agregación (count, sum, avg, min, max) en paralelo.

```
./client.exe -cmd submit -job examples/aggregate_multiple.json
```


3. **complete-pipeline.json** - Pipeline Completo

Pipeline de 4 etapas: read → map → filter → reduce

```
./client.exe -cmd submit -job examples/complete-pipeline.json
```

4. **join_job.json** - Join de Datasets

Join de usuarios y órdenes por user_id.

```
./client.exe -cmd submit -job examples/join_job.json
```

5. **benchmark_job.json** - Benchmark de Performance

Procesa 10K líneas con 3 operadores.

```
./client.exe -cmd submit -job examples/benchmark_job.json
```

Estructura de Archivos y Directorios

```
Proy_2_S0/
├── app/                                # Runtime del sistema
│   ├── data/                          # Archivos de entrada (CSV)
│   │   ├── input.csv
│   │   ├── sales_data.csv
│   │   ├── users.csv
│   │   ├── orders.csv
│   │   ├── large-input.csv (299KB)
│   │   ├── sales_1M.csv (77MB)
│   │   └── wordcount_1M.csv (69MB)
│   ├── results/                      # Archivos de salida generados
│   │   ├── aggregate_count.csv
│   │   ├── aggregate_sum.csv
│   │   ├── aggregate_avg.csv
│   │   ├── benchmark_result-part-0.csv
│   │   └── joined-output.csv
│   ├── temp/                         # Archivos intermedios temporales
│   │   ├── data-read.csv
│   │   ├── mapped.csv
│   │   ├── filtered.csv
│   │   └── spill/                   # Cache overflow
```

```
├── storage/          # Persistencia de estado del Master
│   ├── state-latest.json
│   └── state-1764268712.json
├── scripts/         # Scripts de utilidad
│   └── generate_test_data.py
└── examples/        # Jobs de ejemplo
    ├── aggregate_sales.json
    ├── aggregate_multiple.json
    ├── complete-pipeline.json
    ├── join_job.json
    └── benchmark_job.json
```

Ver Resultados Generados

Listar archivos de resultados

```
ls -lh app/results/
```

Salida:

```
aggregate_avg.csv      66 bytes
aggregate_count.csv    42 bytes
aggregate_sum.csv      66 bytes
benchmark_result.csv   68 KB
joined-output.csv      1.2 KB
```

Ver contenido de un resultado

```
head app/results/aggregate_count.csv
```

Ejemplo de salida:

```
Madrid,3
Barcelona,2
Valencia,2
Sevilla,1
```

Ver estado persistido

```
cat app/storage/state-latest.json | jq .
```

Demo Interactivo

Prueba el sistema con un demo automatizado:

Con Make:

```
make demo
```

Esto ejecuta:

1. Inicia el cluster
2. Espera 5 segundos
3. Ejecuta health check
4. Lista workers registrados
5. Lista jobs existentes

Salida esperada:

```
=== Demo Mini-Spark ===

1. Health Check:
{"master_id":"master-123","status":"healthy","workers_up":3}

2. Workers Registrados:
[{"id":"worker1","status":"UP","last_heartbeat":"2025-11-27T..."}]

3. Lista de Jobs:
[{"id":"job1","status":"SUCCEEDED","tasks":10}]

✓ Demo completado
Ver logs con: make logs
```

Ejecutar Pruebas

Pruebas Unitarias

Con Make:

```
make test
```

Manual:

```
go test ./... -v
```

Pruebas de Integración

Con Make:

```
make test-integration
```

Esto automáticamente:

1. Inicia el cluster
2. Espera 5 segundos
3. Ejecuta pruebas de health check
4. Ejecuta pruebas de API
5. Muestra resultados

Benchmarks de Performance

El proyecto incluye benchmarks formales en la carpeta `benchmarks/`.

Generar Datasets de 1M Registros

```
cd benchmarks  
python3 generate_benchmark_data.py
```

Esto genera:

- `wordcount_1M.csv` (69MB, 1 millón de líneas)
- `sales_1M.csv` (77MB, 1 millón de registros)

Ejecutar Benchmarks**En Linux/Mac:**

```
cd benchmarks  
chmod +x benchmark.sh  
./benchmark.sh
```

En Windows:

```
cd benchmarks
benchmark.bat
```

Ver Reporte de Benchmarks

```
cat benchmarks/BENCHMARK_REPORT.txt
```

Resultados esperados:

- **Word Count 1M:** ~61s (~16,354 líneas/s)
- **Aggregate 1M:** ~36s (~27,708 registros/s)
- **Complex Pipeline 1M:** ~55s (~18,167 registros/s)

Troubleshooting

Problema: Master no responde

Síntomas:

```
curl: (7) Failed to connect to localhost port 8080
```

Solución:

Con Make:

```
# Ver logs del master
make logs-master

# Ver estado del cluster
make status

# Reiniciar el cluster
make restart
```

Con Docker Compose:

```
# Ver logs del master
docker compose logs master

# Verificar que el contenedor esté corriendo
docker compose ps
```

```
# Reiniciar el master
docker compose restart master
```

Problema: Workers no se registran

Síntomas: `workers_up`: 0 en health check

Solución:

Con Make:

```
# Ver logs de workers
make logs-workers

# Ver estado
make status

# Reiniciar
make restart
```

Con Docker Compose:

```
# Ver logs de workers
docker compose logs worker1 worker2 worker3

# Verificar conectividad
docker compose exec worker1 ping master

# Reiniciar workers
docker compose restart worker1 worker2 worker3
```

Problema: Job queda en estado RUNNING

Síntomas: Job no completa después de mucho tiempo

Solución:

```
# Ver detalles del job
./client.exe -cmd status -id <job-id>

# Ver logs para errores (Make)
make logs

# Ver logs para errores (Docker)
docker compose logs -f

# Verificar archivos de entrada
```

```
ls -lh data/

# Reiniciar cluster
make restart    # Con Make
docker compose restart    # Con Docker
```

Problema: Error "file not found"

Síntomas:

```
[ERROR] error abriendo archivo: open /app/data/input.csv: no such file or
directory
```

Solución:

```
# Verificar que el archivo existe en app/data/
ls app/data/input.csv

# Verificar la ruta en el JSON (debe ser relativa, el sistema añade app/data/
automáticamente)
cat examples/my-job.json | grep input_paths
```

Flujo de Trabajo Completo (Ejemplo)

Aquí está un ejemplo end-to-end de usar el sistema:

Opción A: Con Makefile (Recomendado)

```
# 1. Levantar el cluster
make up

# 2. Verificar salud
./client.exe -cmd health

# 3. Listar jobs existentes
./client.exe -cmd list

# 4. Enviar un job de agregación
./client.exe -cmd submit -job examples/aggregate_multiple.json

# 5. Esperar unos segundos
sleep 8

# 6. Ver estado del job
./client.exe -cmd status -id aggregate-multi-function
```

```
# 7. Ver resultados generados
ls -lh app/results/

# 8. Ver contenido de un resultado
cat app/results/aggregate_count.csv

# 9. Ver logs del master
make logs-master

# 10. Limpiar y detener
make down
```

Opción B: Con Docker Compose

```
# 1. Levantar el cluster
docker compose up -d --build

# 2. Esperar a que esté listo
sleep 5

# 3. Verificar salud
./client.exe -cmd health

# 4. Listar jobs existentes
./client.exe -cmd list

# 5. Enviar un job de agregación
./client.exe -cmd submit -job examples/aggregate_multiple.json

# 6. Esperar unos segundos
sleep 8

# 7. Ver estado del job
./client.exe -cmd status -id aggregate-multi-function

# 8. Ver resultados generados
ls -lh app/results/

# 9. Ver contenido de un resultado
cat app/results/aggregate_count.csv

# 10. Ver logs del master
docker compose logs master | tail -20

# 11. Limpiar y detener
docker compose down
```


- **README.md**: Descripción general del proyecto
 - **TODO.md**: Estado de completitud y próximas mejoras
 - **examples/README.md**: Documentación de ejemplos
 - **benchmarks/BENCHMARKS.md**: Guía de benchmarks
 - **docs/ARQUITECTURA.md**: Arquitectura detallada del sistema
-

💡 Tips y Mejores Prácticas

1. **Usa particiones apropiadas**: Para datasets pequeños (<1K registros), usa 1-2 particiones. Para datasets grandes (>100K), usa 4-8 particiones.
2. **Limpia datos intermedios**: Los archivos en `app/temp/` pueden acumularse. Límpialos periódicamente:

```
rm -rf app/temp/*
```

3. **Monitorea el uso de disco**: Los benchmarks de 1M registros generan ~150MB de resultados.
4. **Revisa logs ante errores**: Los logs detallados en `docker compose logs` son la mejor fuente de debugging.
5. **Usa IDs descriptivos**: Da nombres claros a tus jobs para facilitar el tracking:

```
{"id": "daily-sales-aggregation-2025-11-27"}
```

6. **Prueba con datos pequeños primero**: Antes de procesar datasets grandes, valida tu job con 100 líneas.
-

¡Listo para procesar datos distribuidos! 🚀